Universidad del Valle de Guatemala

Curso: Programación orientada a objetos

Segundo Ciclo, 2026



Ejercicio 4

Andrés Eduardo Morataya Oscal - 26497

ANÁLISIS

1. REQUISITOS DEL SISTEMA

RentaMovil requiere un programa de consola para gestionar el alquiler de automóviles, motocicletas y vehículos de carga. Los datos se mantendrán en memoria mientras el programa esté funcionando, sin utilizar archivos ni bases de datos.

El sistema debe ofrecer las siguientes funciones:

- Registrar vehículos: solicitar la información general y las características de cada categoría. La placa debe tener contenido y no puede coincidir con la de otro vehículo registrado. Todo vehículo nuevo debe comenzar como disponible.
- Consultar la flota: mostrar los datos, las características, la categoría y el estado de disponibilidad de todos los vehículos.
- Buscar por placa: localizar un vehículo sin diferenciar entre letras mayúsculas y minúsculas, y omitiendo los espacios al inicio y al final de la placa.
- Cotizar un alquiler: solicitar una placa registrada y un número entero positivo de días. Debe mostrar las características del vehículo, su disponibilidad y el precio total. También se pueden cotizar vehículos que estén alquilados.
- Alquilar un vehículo: verificar que exista y esté disponible, presentar el costo y pedir confirmación. El ingreso y el cambio de disponibilidad se registran únicamente cuando el usuario acepta.
- Cancelar la operación: volver al menú sin modificar los ingresos ni el estado del vehículo.
- Registrar una devolución: verificar que el vehículo exista y esté alquilado. Luego, marcarlo como disponible, sin realizar cobros adicionales ni reducir los ingresos.
- Consultar el reporte: presentar el total de vehículos, la cantidad registrada, disponible y alquilada por categoría, y los ingresos acumulados.
- Salir: cerrar el programa mediante una opción del menú.

Al comenzar, el programa debe cargar dos vehículos por categoría, para un total de seis. Todos estarán disponibles y los ingresos iniciales serán de Q0.00. Los ejemplos deben incluir automóviles mecánicos y automáticos, motocicletas con y sin recargo, y vehículos de carga con capacidades decimales.

Si el usuario introduce datos con un formato incorrecto, el sistema debe mostrar un mensaje y permitir que continúe. Los valores monetarios se presentarán con dos decimales.

2. DATOS Y VALIDACIONES

La información que utilizará el sistema debe cumplir estas condiciones:

- Placa: dato de tipo String. Es obligatoria y debe ser única en toda la flota.
- Marca: dato de tipo String. No se permite un valor nulo ni una cadena vacía.
- Modelo: dato de tipo String. No se permite un valor nulo ni una cadena vacía.
- Tarifa diaria: dato de tipo double. Debe ser mayor que cero.
- Disponibilidad: dato de tipo boolean. Comienza en true y se modifica al alquilar o devolver el vehículo.
- Cantidad de pasajeros: dato de tipo int. Debe ser mayor que cero.
- Transmisión automática: dato de tipo boolean. El valor true identifica una transmisión automática y false una mecánica.
- Cilindraje: dato de tipo int. Se expresa en centímetros cúbicos y debe ser mayor que cero.
- Capacidad máxima de carga: dato de tipo double. Se expresa en toneladas y debe ser mayor que cero.
- Categoría: dato de tipo String. Se establece en el constructor de cada clase hija.
- Días de alquiler: dato de tipo int. Debe ser mayor que cero y se solicita al cotizar o alquilar.
- Ingresos acumulados: dato de tipo double. Comienza en cero y aumenta solamente cuando se confirma un alquiler.

Los días de alquiler se utilizan como entrada para calcular el precio, pero no se guardan como atributo del vehículo. Esta versión no contempla fechas, información de clientes, reservaciones, multas ni un historial de alquileres.

3. REGLAS PARA CALCULAR EL COSTO

El precio depende de la categoría y de las características del vehículo:

- Automóvil mecánico: costo total = tarifa diaria × días.
- Automóvil automático: costo total = (tarifa diaria + Q50.00) × días.
- Motocicleta de hasta 250 cc: costo total = tarifa diaria × días.
- Motocicleta de más de 250 cc: costo total = (tarifa diaria × días) + Q75.00.

- Camión de carga: costo total = (tarifa diaria + Q100.00 × capacidad máxima en toneladas) × días.

En las motocicletas de más de 250 cc, el recargo de Q75.00 se cobra una sola vez por alquiler. Para los vehículos de carga, el recargo se calcula según su capacidad máxima, independientemente de la carga que realmente transporten.

Ejemplo de cálculo:

Un vehículo de carga tiene una tarifa diaria de Q200.00, una capacidad máxima de 1.5 toneladas y se alquila durante tres días.

- Costo base: Q200.00 × 3 = Q600.00.
- Recargo por capacidad: Q100.00 × 1.5 × 3 = Q450.00.
- Total del alquiler: Q600.00 + Q450.00 = Q1,050.00.

4. CLASES Y RESPONSABILIDADES

Clase Vehiculo

Es una clase abstracta que agrupa los atributos compartidos por todos los vehículos: placa, marca, modelo, tarifa diaria y disponibilidad. Su constructor recibe y valida los datos generales, y establece la disponibilidad inicial en true.

Los atributos son privados y se consultan mediante getters. El método validar(int dias) verifica que la cantidad de días sea positiva, mientras que obtenerDetalles() devuelve la información común del vehículo.

Los métodos quitarDisponibilidad() y habilitarDisponibilidad() permiten cambiar su estado.

También declara los métodos abstractos calcularTarifa(int dias) y getCategoria(), que cada clase hija debe implementar.

Clase Automovil

Hereda de Vehiculo e incorpora la cantidad de pasajeros, el tipo de transmisión y la categoría. Su constructor comprueba que la cantidad de pasajeros sea positiva y asigna la categoría "Automovil".

Para calcular el alquiler, añade Q50.00 por día si la transmisión es automática. Además, amplía obtenerDetalles() para mostrar los pasajeros, la transmisión y la categoría.

Clase Moto

Hereda de Vehiculo e incorpora el cilindraje y la categoría "Motocicleta". Su constructor verifica que el cilindraje sea mayor que cero.

Al calcular el precio, aplica un recargo único de Q75.00 cuando el cilindraje supera los 250 cc. Sus detalles incluyen la información general, el cilindraje y la categoría.

Clase Camion

Hereda de Vehiculo y añade la capacidad máxima en toneladas y la categoría "Camion de carga". Esta clase representa la camioneta de carga mencionada en el enunciado.

Su constructor comprueba que la capacidad sea positiva. El cálculo del alquiler agrega Q100.00 por tonelada de capacidad por cada día. La capacidad también se muestra en los detalles del vehículo.

Clase Renta

Gestiona una colección ArrayList<Vehiculo> y el total de ingresos acumulados. Su constructor crea la lista vacía e inicia los ingresos en cero.

Se encarga de buscar vehículos, impedir placas duplicadas, consultar la flota, realizar cotizaciones, registrar alquileres y devoluciones, y elaborar el reporte.

El método buscarPlaca() devuelve el vehículo encontrado o null cuando no existe una coincidencia.

Los conteos del reporte se obtienen recorriendo la flota, sin mantener contadores adicionales que puedan quedar desactualizados.

Clase Main

Contiene el punto de inicio del programa. Crea una instancia de Renta, registra los seis vehículos iniciales y mantiene visible el menú hasta que el usuario elige salir.

Utiliza Scanner para recibir datos, presenta los resultados, solicita confirmaciones y captura las excepciones generadas por el modelo. También incluye métodos auxiliares para leer textos, números enteros, números decimales y respuestas de confirmación.

5. PROCESO DE ALQUILER

El alquiler se realiza mediante la siguiente secuencia:

1. Main solicita la placa y busca el vehículo mediante Renta.buscarPlaca().
2. Se verifica que el vehículo exista y esté disponible.
3. Main pide la cantidad de días y llama a Renta.cotizar().
4. Renta ejecuta calcularTarifa(dias) sobre el vehículo seleccionado.
5. La clase correspondiente valida los días y aplica su regla de cobro.
6. Main muestra las características y el precio total, y solicita la confirmación del usuario.
7. Si el usuario acepta, Main llama a Renta.alquilar(). Este método comprueba nuevamente la disponibilidad, calcula el costo, marca el vehículo como alquilado y suma el importe a los ingresos.
8. Si el usuario cancela, la operación termina sin modificar los datos.

La herencia permite reunir en Vehiculo los atributos y comportamientos compartidos. Así, cada clase hija se concentra en sus características y reglas de cobro particulares.

Clase abstracta: Vehiculo representa lo común y exige que cada clase hija implemente calcularTarifa(int dias) y getCategoria().

Características particulares: pasajeros y transmisión pertenecen a Automovil; cilindraje a Moto; capacidad de carga a Camion. Así evitas que un vehículo tenga atributos que no le corresponden.

Polimorfismo: ArrayList<Vehiculo> permite gestionar distintas categorías y llamar a calcularTarifa(dias); cada objeto ejecuta su propia regla de cobro.

Nueva categoría: habría que crear otra clase que herede de Vehiculo, validar sus atributos e implementar sus métodos. También se modificaría el menú de registro en Main. La búsqueda, las devoluciones y la gestión general podrían mantenerse. El reporte necesitaría ajustes si sus categorías están escritas de forma fija; si las agrupa mediante getCategoria(), podría incluir la nueva categoría sin modificar esa lógica.